Experience-Weighted Attraction

General Principles

Social learning models often aim to disentangle asocial learning (learning from personal experience) from social learning (learning from others). The **Experience-Weighted Attraction (EWA)** model (**McElreath2018?**) provides a robust framework for this. In the context of the Panama capuchin monkey study Barrett, McElreath, and Perry (2017), the model tracks how individuals update their "attractions" to different foraging techniques based on their own success and the social cues they observe from others.

The core idea is that individuals have a set of attractions $A_{i,j,t}$ for each technique j. These attractions are updated over time, and the probability of choosing a technique is a mixture of asocial preferences (derived from attractions) and social influence.

In this documentation, we present two primary variations of the EWA model adapted from Barrett, McElreath, and Perry (2017):

1. **Social Learning (Combo)**: A streamlined version focusing on the interplay between individual experience (asocial learning) and social frequency biased by payoffs. It uses a reduced parameter set (4 varying effects) to identify the core drivers of behavioral diffusion.
2. **Social Learning (Global Age)**: A comprehensive hierarchical model that evaluates multiple social biases (e.g., kinship, prestige, age similarity) and explicitly models how an individual's age influences their learning rate (ϕ) and reliance on social cues (γ).

---

1. Social Learning (Combo)

The "Combo" model is a basic version that includes asocial learning and pay-off biased social learning. It evaluates four varying effects: learning rate, social weight, conformity, and pay-off bias.

Example

> **Note**
>
> - **Vectorization**: The model uses `jax.lax.scan` for efficient sequential likelihood calculation over foraging bouts. This is critical for performance in JAX-based backends like BI.
> - **Hierarchical Structure**: Intercepts for learning rates and social influence vary by individual (`id`). Barrett, McElreath, and Perry (2017) use a Cholesky-factored multivariate normal prior to account for correlations between these effects.
> - **Dtype Sensitivity**: Ensure all data arrays are `float64` to match JAX's 64-bit backend requirements for numerical stability.

Python

```python
from BI import bi, jnp
import jax

m = bi(platform='cpu')

def model(K, J, tech, id, bout, y_prev, s, ps):
    # Priors
    lam   = m.dist.exponential(name="lambda", rate=1.0)
    mu    = m.dist.normal(name="mu", loc=jnp.zeros(4), scale=1.0)
    sigma = m.dist.exponential(name="sigma", rate=jnp.full(4, 3.0))
    L_Rho = m.dist.lkj_cholesky(name="L_Rho", dimension=4, concentration=3.0)
    z     = m.dist.normal(name="z", loc=jnp.zeros((4, J)), scale=1.0)

    # Varying effects
    L_scaled = L_Rho * sigma[:, None]
    a_id     = (L_scaled @ z).T

    # Parameters per individual
    phi_arr   = jax.nn.sigmoid(mu[0] + a_id[id, 0])      # Learning rate
    gamma_arr = jax.nn.sigmoid(mu[1] + a_id[id, 1])      # Social weight
    fconf_arr = jnp.exp(mu[2] + a_id[id, 2])             # Conformity
    Bpay_arr  = mu[3] + a_id[id, 3]                      # Pay-off bias

    # Scan carry: (Attractions, Log-likelihood)
    def step_fn(carry, x):
        AC, ll = carry
```

```
        (phi_i, gamma_i, fconf_i, Bpay_i, tech_i, id_i, bout_i, prev_y_i, s_i, ps_i) = x

        # Update attractions (EWA)
        ac_new = jnp.where(bout_i == 1, jnp.zeros(K), (1.0 - phi_i) * AC[id_i] + phi_i * prev

        # Asocial choice probability
        logPrA = (lam * ac_new)[tech_i] - jax.nn.logsumexp(lam * ac_new)

        # Social choice probability (Pay-off bias)
        lin_mod = jnp.concatenate([jnp.ones(1), jnp.exp(Bpay_i * ps_i[1:])]) * jnp.power(s_i
        PrS = lin_mod[tech_i] / jnp.sum(lin_mod)

        # Mixture likelihood
        prob_mix = (1.0 - gamma_i) * jnp.exp(logPrA) + gamma_i * PrS
        lik_i = jnp.where(bout_i > 1, jnp.log(prob_mix), logPrA)

        return (AC.at[id_i].set(ac_new), ll + lik_i), None

    AC_init = jnp.zeros((J, K))
    xs = (phi_arr, gamma_arr, fconf_arr, Bpay_arr, tech, id, bout, y_prev, s, ps)
    (_, total_ll), _ = jax.lax.scan(step_fn, (AC_init, 0.0), xs)

    m.dist.factor("obs", total_ll)

m.fit(model)
m.summary()
```

R

```
library(BayesianInference)
m <- importBI(platform='cpu')

model <- function(K, J, tech, id, bout, y_prev, s, ps) {
  # Priors
  lam <- m$dist$exponential(name="lambda", rate=1.0)
  mu <- m$dist$normal(name="mu", loc=jnp$zeros(4L), scale=1.0)
  sigma <- m$dist$exponential(name="sigma", rate=jnp$full(4L, 3.0))
  L_Rho <- m$dist$lkj_cholesky(name="L_Rho", dimension=4L, concentration=3.0)
  z <- m$dist$normal(name="z", loc=jnp$zeros(tuple(4L, J)), scale=1.0)

  # ... Equivalent logic to Python implementation ...
```

```
}
m$fit(model)
m$summary()
```

Julia

```
using BayesianInference
m = importBI(platform="cpu")

@BI function model(K, J, tech, id, bout, y_prev, s, ps)
    # Priors
    lam = m.dist.exponential(name="lambda", rate=1.0)
    mu = m.dist.normal(name="mu", loc=jnp.zeros(4), scale=1.0)
    sigma = m.dist.exponential(name="sigma", rate=jnp.full(4, 3.0))
    L_Rho = m.dist.lkj_cholesky(name="L_Rho", dimension=4, concentration=3.0)
    z = m.dist.normal(name="z", loc=jnp.zeros(4, J), scale=1.0)

    # ... Equivalent logic to Python implementation ...
end
m.fit(model)
m.summary()
```

Mathematical Details

The "Combo" model links individual learning rates and social frequency to behavioral choice.

Attraction Update:

$$A_{j,t+1} = (1 - \phi_j)A_{j,t} + \phi_j p_{j,t}$$

Softmax Choice:

$$I_{j,t} = \text{Softmax}(\lambda \cdot A_{j,t})$$

Social Probability:

$$S_{j,t} = \frac{N_{j,t}^f \exp(\beta_{\text{pay}} \cdot p_{j,t})}{\sum_k N_{k,t}^f \exp(\beta_{\text{pay}} \cdot p_{k,t})}$$

---

2. Social Learning (Global Age)

The "Global Age" model expands the Combo version by incorporating a wider array of social biases (kinship, prestige, age similarity, etc.) and modeling the linear influence of individual age on the learning parameters ϕ and γ.

Example

Python

```
from BI import bi, jnp
import jax

m = bi(platform='cpu')

def model(K, J, tech, id, bout, y_prev, s, ps, ks, pr, co, yo, age):
    # Priors (8 varying effects)
    lam   = m.dist.exponential(name="lambda", rate=1.0)
    mu    = m.dist.normal(name="mu", loc=jnp.zeros(8), scale=1.0)
    sigma = m.dist.exponential(name="sigma", rate=jnp.full(8, 3.0))
    L_Rho = m.dist.lkj_cholesky(name="L_Rho", dimension=8, concentration=3.0)
    z     = m.dist.normal(name="z", loc=jnp.zeros((8, J)), scale=1.0)
    b_age = m.dist.normal(name="b_age", loc=jnp.zeros(2), scale=1.0)

    # Varying effects
    L_scaled = L_Rho * sigma[:, None]
    a_id     = (L_scaled @ z).T

    # Parameters per individual with age effects
    phi_arr   = jax.nn.sigmoid(mu[0] + a_id[id, 0] + b_age[0] * age)
    gamma_arr = jax.nn.sigmoid(mu[1] + a_id[id, 1] + b_age[1] * age)
    fconf_arr = jnp.exp(mu[2] + a_id[id, 2])
    B_pay     = mu[3] + a_id[id, 3]
    B_kin     = mu[4] + a_id[id, 4]
    B_pres    = mu[5] + a_id[id, 5]
    B_coho    = mu[6] + a_id[id, 6]
    B_yob     = mu[7] + a_id[id, 7]

    def step_fn(carry, x):
        AC, ll = carry
        (phi_i, gamma_i, fconf_i, Bp, Bk, Bpr, Bc, By, t_i, id_i, b_i, py_i, s_i, ps_i, ks_i
```

```
        ac_new = jnp.where(b_i == 1, jnp.zeros(K), (1.0 - phi_i) * AC[id_i] + phi_i * py_i)
        logPrA = (lam * ac_new)[t_i] - jax.nn.logsumexp(lam * ac_new)

        # Multi-cue social influence
        lin_rest = jnp.exp(Bp * ps_i[1:] + Bk * ks_i[1:] + Bpr * pr_i[1:] + Bc * co_i[1:] + B
        lin_mod = jnp.concatenate([jnp.ones(1), lin_rest]) * jnp.power(s_i, fconf_i)
        PrS = lin_mod[t_i] / jnp.sum(lin_mod)

        prob_mix = (1.0 - gamma_i) * jnp.exp(logPrA) + gamma_i * PrS
        lik_i = jnp.where(b_i > 1, jnp.log(prob_mix), logPrA)
        return (AC.at[id_i].set(ac_new), ll + lik_i), None

    AC_init = jnp.zeros((J, K))
    xs = (phi_arr, gamma_arr, fconf_arr, B_pay, B_kin, B_pres, B_coho, B_yob,
          tech, id, bout, y_prev, s, ps, ks, pr, co, yo)
    (_, total_ll), _ = jax.lax.scan(step_fn, (AC_init, 0.0), xs)
    m.dist.factor("obs", total_ll)

m.fit(model)
m.summary()
```

R

```
# Equivalent R implementation with 8 effects and age slopes...
```

Julia

```
# Equivalent Julia implementation with 8 effects and age slopes...
```

Mathematical Details

The "Global Age" model incorporates the linear influence of individual traits on learning parameters.

Age-Dependent Parameters:

$$\text{logit}(\phi_j) = \alpha_{\phi,j} + \beta_\phi \cdot \text{age}_j$$

$$\text{logit}(\gamma_j) = \alpha_{\gamma,j} + \beta_\gamma \cdot \text{age}_j$$

Multi-Cue Social Component:

$$B_{j,t} = \sum_{k \in \{\text{pay, kin, rank, sim, bias}\}} \beta_k \kappa_{k,j,t}$$

The realized choice probability follows the same mixture rule as the Combo model but with expanded social cue integration.

Notes

> **Note**
>
> - The `Global Age` model is significantly more complex and requires larger sample sizes to resolve the correlations between varying effects.
> - Cholesky decomposition of the correlation matrix is used for both models to ensure positive-definiteness.

Reference(s)

Barrett, Brendan J., Richard L. McElreath, and Susan E. Perry. 2017. "Pay-Off-Biased Social Learning Underlies the Diffusion of Novel Extractive Foraging Traditions in a Wild Primate." *Proceedings of the Royal Society B: Biological Sciences* 284 (1856): 20170358. https://doi.org/10.1098/rspb.2017.0358.